

PRÁCTICA 3: Sockets – GRUPAL

Asignatura:	Programación de Sistemas Distribuidos
Curso:	2023/2024
Fecha:	12-04-2024
Semestre:	2º
Generado:	2026-05-04 13:10

Repositorio base: chat -example (Socket.IO)

Grupo: (2-3 personas)

Despliegue en Replit. Documenta los pasos (1 punto)

1) Crear Repl

Entrar en Replit y elegir **Create Repl**.

Opción recomendada: **Import from GitHub**.

Pegar el repositorio (o el fork) y crear el repl.

2) Instalar dependencias

Replit detecta `package.json` y ejecuta `npm install` automáticamente.

Si no lo hace, ejecutarlo manualmente desde la consola: `npm install`.

3) Configurar el comando de ejecución

En este proyecto el arranque es:

```
npm start (script "start": "node index.js")
```

Alternativa: usar directamente `node index.js`.

El fichero `.replit` ya define:

```
run = "node index.js"
```

4) Puerto

El servidor usa `process.env.PORT || 3000`.

En Replit se proporciona PORT automáticamente; no hace falta tocar el código.

5) Probar

Abrir la URL pública del repl.

Pulsar **Conectar**, enviar mensajes, y verificar que:

Se reciben mensajes en tiempo real.

El botón **Desconectar** cierra el socket.

Tras desconectar, el input y el botón de envío quedan deshabilitados.

Dependencias de la aplicación y breve descripción (1 punto)

En `package.json`:

`express`

Framework HTTP para Node.js.

Se usa para servir la ruta `/` y entregar el `index.html`.

`socket.io`

Librería de comunicación bidireccional en tiempo real (cliente/servidor).

Abstrae WebSocket y añade reconexión, eventos y “fallbacks”.

Se usa para emitir y recibir eventos como `chat message`.

Principales ficheros de la aplicación y su función (2 puntos)

`index.js`

Arranca Express + servidor HTTP.

Inicializa Socket.IO sobre el servidor HTTP.

Gestiona conexiones (`connection`) y eventos:

- `chat message`: recibe mensajes y los re-emite a todos.

- (modificación) `set username`: asigna nombre de usuario por socket.

- (modificación) `disconnect`: notifica la salida y actualiza presencia.

`index.html`

Interfaz del chat (lista de mensajes + formulario).

Cliente Socket.IO (`/socket.io/socket.io.js`).

(modificación) Botones **Conectar/Desconectar**, estado de conexión y bloqueo del envío cuando no hay socket.

`package.json`

Dependencias y script de arranque (`npm start`).

`replit`

Define el comando “Run” en Replit (`node index.js`).

Botón para cerrar conexión del socket y bloquear envío (2 puntos)

Se ha implementado en `index.html`:

Botón “Desconectar”

Ejecuta `socket.disconnect()` para cerrar el socket.

Bloqueo de envío

Cuando el socket no está conectado:

```
input.disabled = true
```

```
send.disabled = true
```

El submit del formulario ignora el envío si `!socket || !socket.connected`.

Modificaciones extra (4 puntos)

Se han añadido mejoras para completar la parte opcional:

Control explícito de conexión

El usuario decide cuándo conectar (botón **Conectar**).

Se muestra un **estado** (“Conectado/Desconectado”).

Nombre de usuario

Se pide una vez (prompt) y se guarda en `localStorage`.

Se envía al servidor con el evento `set username`.

Mensajes de sistema

Entrada/salida de usuarios y mensajes informativos (con estilo diferente).

Indicador de presencia: “Usuarios conectados: N”.